

PCU-Select: Amortizing Target-Specific Data Selection Across PEFT Configurations

Anonymous submission

Abstract

A large language model (LLM) serving system typically maintains a registry of fine tuning configurations to serve diverse tasks and deployment budgets. These configurations vary in which modules are trained and how they are updated, leading to different trainable subspaces. This diversity poses a challenge for gradient based data selection: it must rebuild a target specific gradient datastore for every configuration, making cost scale linearly with registry size. Reusing a single subset across all configurations avoids this cost, but degrades quality because example utility is tightly coupled to the trainable subspace. We present PCU Select, which amortizes data selection across a configuration registry by conditioning utility on the task and a structured code that encodes the fine tuning strategy. The key insight is to decouple gradient recomputation from the target configuration through 24 shared intervention sites across the Transformer backbone. These sites provide a common basis for comparing different fine tuning methods. PCU Select then extracts reusable representations offline, generates multi fidelity utility labels using cheap gradient approximations calibrated by short fine tuning runs, and trains a conditional scorer to jointly predict utility and uncertainty. At deployment, the scorer ranks cached features for any target configuration and task without rebuilding gradients. On Llama 2 7B across four tasks and five configurations, PCU Select reduces the cost of onboarding a new configuration by 44 times compared to per configuration LESS, from 63.2 to 1.44 GPU hours, while maintaining downstream quality within a negligible margin of the baseline. By amortizing per configuration computation into a one time offline stage, PCU Select makes gradient level data selection practical at registry scale.

1 Introduction

Large language model platforms must adapt one backbone to different tasks and resource limits. Parameter-efficient fine-tuning (PEFT) reduces adaptation cost by updating only a small set of model parameters, allowing a platform to maintain a *registry* of configurations that differ in method, trainable modules, capacity, or training recipe. These trainable parameters determine a configuration’s *trainable subspace*: the

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

update directions available during fine-tuning. Each training example can affect the model only through the directions that the target configuration exposes. Data selection reduces cost along a complementary axis by choosing a compact subset whose examples produce useful task updates. Its decisions therefore depend on the target trainable subspace: changing the subspace can change which examples the selector should prioritize.

Most data selectors ignore this interaction. Quality filters, diversity heuristics, and proxy-model scores produce one ranking for a task and reuse it across PEFT configurations. We call such a ranking *PEFT-agnostic* because the selector does not receive the target configuration. Yet common PEFT methods permit different updates: LoRA adds low-rank changes to selected projections (Hu et al. 2022); adapters insert trainable bottlenecks (Houlsby et al. 2019); and (IA)³ scales selected activations (Liu et al. 2022). The same example can therefore produce different updates, and hence have different value, under different configurations.

Gradient-based selectors capture this dependence but incur repeated computation. LESS (Xia et al. 2024) first builds a target-specific *gradient datastore*, a cache containing a training gradient for each candidate example. It then ranks the candidates against a *task sketch*, a small set of training or development examples that represents the target task. Because each PEFT configuration exposes different trainable parameters, LESS must rebuild the cache for every configuration. Its selection cost therefore grows linearly with the registry.

One shortcut is to reuse a single LESS subset for every configuration. This reuse avoids repeated computation but removes the configuration information that makes gradient selection useful. In our evaluation, the shortcut performs worse than a PEFT-agnostic representation baseline (Table 2). Existing methods thus force a choice: recompute a target-specific ranking for every configuration or reuse a ranking that ignores the configuration.

The configuration dependence affects downstream performance as well as the rankings. Figure 1 shows that independent runs for the same configuration agree more closely than runs across configurations. Changes within one PEFT family preserve more of the ranking, while cross-family changes produce the largest disagreement. When selection and training use the same PEFT, the resulting subsets outperform